

Séance 2

29/05 - 8h

Déclaration de variables:

```
var a string = "a"
```

```
var a = "a"
```

```
a := "a"
```

Ce que j'ai appris pendant l'exercice :

```
fmt.Print("Entrez votre poids (en kg, ex: 70.5) : ")
```

```
_, err := fmt.Scanln(&poids)
```

fmt.Print pour faire un input en console

fmt.Scanln pour attribuer la valeur entrée à une variable

Les fonctions

Surcharge impossible

Peut retourner plusieurs valeurs (résultat et erreur)

→ permet de gérer les erreurs en go (remplace les try catch)

```
resultat, err := diviser (10,3)
if err != nil {...}
```

Variadic = avec un nb variable d'arguments (ex: fonction println)

Les **closures** sont un concept puissant en programmation qui permet de capturer et de se souvenir des variables de leur environnement.

Boucles

le for fait tout (for, foreach, while ...)

Ce que j'ai appris pendant l'exercice 2:

Gérer les erreurs

```
if err != nil {  
    fmt.Printf("Erreur : %v\n", err)  
    continue  
}
```

Fonction variadique

```
func creerOperation(op string) func(float64, float64) float  
64 {
```

Panic (permet d'interrompre l'exécution normale d'une goroutine)

```
panic("opérateur inconnu")
```

Fallthrough: The fallthrough keyword in Golang is ***used within a case block of a switch statement*** to transfer control to the next case block.

Slice = sac à dos qui peut s'agrandir

slice := make([]int, 3, 5) // longueur 3, capacité 5

append = ajoute à la fin (étend)

map → dictionnaire cle/valeur

copy des slices

Slices carry a small header that can be duplicated, but the backing array behavior changes the outcome depending on how you copy. Assigning one

slice to another is different from using the `copy` function, and slicing creates yet another pattern of shared storage.

<https://medium.com/@AlexanderObregon/go-slice-copy-mechanics-4e7472f4f730>

Structure

Positionnelle

- `produit := Produit{iphone, 99, 50}`

Nommée

- `produit := Produit{name: iphone, size: 99 ...}`

Affectée

- `produit := Produit {`
- `produit.name = iphone ...`

* permet de modifier vraiment l'objet sans avoir à l'affecter

→ pointer receiver, go fait & automatiquement

Embedding : on assemble des pièces pour créer des choses plus complexes

```

// Type de base
type Adresse struct {
    Rue      string
    Ville    string
    CodePostal string
    Pays     string
}

func (a Adresse) Format() string {
    return fmt.Sprintf("%s, %s %s, %s", a.Rue, a.CodePostal, a.Ville, a.Pays)
}

type Personne struct {
    Prenom string
    Nom    string
    Age    int
}

func (p Personne) NomComplet() string {
    return p.Prenom + " " + p.Nom
}

// Client CONTIENT une Personne et une Adresse (embedding)
// Syntaxe : le type seul, sans nom de champ
type Client struct {
    Personne // embedding : toutes les méthodes de Personne sont promues
    Adresse  // embedding : toutes les méthodes d'Adresse sont promues
    Email    string
    NumeroClient int
}

// Utilisation
c := Client{
    Personne: Personne{Prenom: "Alice", Nom: "Martin", Age: 30},
    Adresse:  Adresse{Rue: "12 rue de la Paix", Ville: "Paris",
        CodePostal: "75001", Pays: "France"},
    Email:    "alice@example.com",
    NumeroClient: 1001,
}

```

Promotion : je peux accéder à c.Prenom

Tag : chaines entre ``

```
// Les tags sont des chaînes entre backticks après chaque champ
type Utilisateur struct {
    ID      int    `json:"id"`           // sérialisé comme "id"
    Prenom  string  `json:"prenom"`
    Email   string  `json:"email,omitempty"` // omis si vide ("")
    Age     int    `json:"age,omitempty"`   // omis si 0
    Interne string  `json:"- "`       // JAMAIS sérialisé
    MDP     string  `json:"- "`       // mot de passe : jamais !
}

// Exemple avec JSON (anticipation Jour 3)
import "encoding/json"

u := Utilisateur{ID: 1, Prenom: "Alice", Email: "alice@go.dev", Age: 30}
data, _ := json.Marshal(u)
fmt.Println(string(data))
// {"id":1,"prenom":"Alice","email":"alice@go.dev","age":30}

// Unmarshal : JSON → struct
jsonStr := `{"id":2,"prenom":"Bob"}`
var u2 Utilisateur
json.Unmarshal([]byte(jsonStr), &u2)
fmt.Println(u2.Prenom) // Bob
```

Ce que j'ai appris dans l'exercice 3

Les différentes boucles for

```
for _, emp := range employes {
    fmt.Println(emp.FicheEmploye() + "\n")
}
```

L'embeded

```
type Personne struct {
    Prenom string
    Nom    string
    Age    int
    Email  string
}

type Etudiant struct {
    Personne
    Promo  string
}
```

```
Moyenne float64
```

```
}
```

29/05 - 14h

Accès aux attributs : en fonction de la casse

Majuscule = exporté

Camel Case débute avec une minuscule : privé

Pascal Case débute par une majuscule : partagé

Unlike other program languages like Java and Python that use *access modifiers* such as `public`, `private`, or `protected` to specify scope, Go determines if an item is `exported` and `unexported` through how it is declared. Exporting an item in this case makes it `visible` outside the current package. If it's not exported, it is only visible and usable from within the package it was defined.

This external visibility is controlled by capitalizing the first letter of the item declared. All declarations, such as `Types`, `Variables`, `Constants`, `Functions`, etc., that start with a capital letter are visible outside the current package.

<https://www.digitalocean.com/community/tutorials/understanding-package-visibility-in-go>

defer = post-it à la porte de sortie → pour fermer les fichiers, libérer les ressources ...

garanti même si la fonction plante

```
// Ordre LIFO – important pour les ressources dépendantes
func exemple() {
    fmt.Println("début")
    defer fmt.Println("3 – dernier defer, premier exécuté")
    defer fmt.Println("2")
    defer fmt.Println("1 – premier defer, dernier exécuté")
    fmt.Println("fin")
}
// Sortie : début → fin → 1 → 2 → 3
```

Packages standards

fmt

strings

strconv

math

sort

time

Ce que j'ai appris dans le tp :

bufio avec os.Stdin

strings.EqualFold pour la comparaison de chaîne

slices.IndexFunc pour la recherche dans un slice